

IIE — Intelligent Integration Engine

Technical Report (Thesis Companion)

Project: OHCS Process Intelligence Platform **Stack:** Cloudflare Workers · D1 (SQLite) · Workers AI · Vectorize · R2 · React/TypeScript **Status:** Phases 1–4 implemented and validated (6/6 PRD success metrics, 32 automated tests)

1. Introduction

1.1 Background

The Office of the Head of Civil Service (OHCS) in Ghana operates nine organisational units — five directorates (RSIMD, PBMED, CMD, F&A, RTDD) and four units (RCU, CSC, IAU, PR) — with a staff strength of 150 officers structured along Civil Service grades: each unit is headed by a Director, supported by middle management running from Assistant Director I up to Deputy Director, with officers of lower grades beneath them. Day-to-day operations generate digital footprints across several disconnected subsystems: RFID-based attendance capture, a leave administration workflow, and an HR chatbot.

These subsystems record *what happened*, but nobody can see *how work actually flows*. Management questions such as "where do leave requests stall?", "do approvals follow the prescribed chain?", and "which directorate struggles with punctuality?" are today answered — if at all — through manual audits and anecdote.

1.2 Problem statement

OHCS lacks (a) a unified record of operational events across its subsystems, and (b) any automated means of turning such a record into process insight. The consequence is invisible bottlenecks, undetected policy deviations, and decision-making without evidence.

1.3 Aim and objectives

Aim: design and implement an event-driven process intelligence platform that discovers, monitors, and analyses OHCS business processes from a unified event log.

Objectives:

1. Define a canonical event model and a unified event log ingesting attendance, leave-workflow, and chatbot events.
2. Implement process discovery that reconstructs the actual workflow (directly-follows graph + variants) without prior configuration.
3. Implement bottleneck analysis (transition-duration statistics with configurable SLA flagging).
4. Implement conformance checking against the prescribed leave-approval chains, including the F&A/RTDD routing split.
5. Provide decision-support recommendations and cross-department comparisons for management.
6. Provide an AI assistant answering HR policy questions via retrieval-augmented generation and executing leave transactions safely.
7. Validate the platform against measurable PRD success criteria using a seeded ground truth.

1.4 Scope

The platform covers event ingestion, storage, mining, analysis, and presentation for the three subsystems above, within a single Cloudflare Worker. Real subsystem integration is simulated by a deterministic synthetic data generator; per-user identity (SSO) and queue-based ingestion fanout are deferred to later phases.

2. Literature and theoretical foundations

2.1 Process mining

Process mining bridges data science and process science by extracting process knowledge from event logs (van der Aalst, *Process Mining: Data Science in Action*, 2016). Its three classical forms map directly onto this platform's features:

- **Discovery** — learn a model from the log without a priori information. IIE builds a **directly-follows graph (DFG)**: activities as nodes, observed hand-offs as edges, annotated with frequencies (§6.1 of the PRD).
- **Conformance checking** — compare a prescribed model against reality. IIE replays each leave trace against the prescribed chain and reports *skipped steps* and *out-of-order execution* with a fitness-style score.
- **Enhancement** — extend a model with performance information. IIE overlays transition-duration statistics (median, P95) onto the discovered map and flags steps breaching SLA thresholds.

Edge reliability uses the **dependency measure** popularised by the Heuristic Miner (Weijters & van der Aalst): for activities a, b with directly-follows counts $|a \rightarrow b|$ and $|b \rightarrow a|$,

$$\text{dependency}(a,b) = (|a \rightarrow b| - |b \rightarrow a|) / (|a \rightarrow b| + |b \rightarrow a| + 1)$$

which is symmetric-robust and bounded in $(-1, 1)$. The event model follows the spirit of the IEEE **XES** standard (1849-2016): cases (traces), events with activity, timestamp, resource, and payload attributes.

2.2 Workflow modelling

The leave process is modelled as an explicit **finite state machine** (submitted → review → verification → approval → terminal). Notably, the *same* machine that enforces live transitions supplies the prescribed model for conformance checking — one source of truth for enforcement and audit. Leave administration forks by type: standard leave is verified by the F&A admin officer and approved by Director F&A; study leave (across OHCS and the wider Civil Service) is reviewed by the RTDD Schedule Officer and approved by Director RTDD.

2.3 Retrieval-augmented generation

The assistant applies **RAG** (Lewis et al., NeurIPS 2020): policy documents are chunked (~800 characters, paragraph-aligned), embedded (`bge-base-en-v1.5`), and indexed in a vector database (Vectorize). At query time the top-K chunks ($K=3$, cosine similarity) ground a large language model (`llama-3.3-70b-instruct-fp8-fast`), which answers *strictly from retrieved excerpts*, suppressing hallucination. A similarity floor (0.45) triggers deflection to HR when the corpus is silent.

2.4 Serverless edge architecture

The platform adopts a serverless model on Cloudflare Workers: compute, a SQL database (D1, SQLite), vector index, and object storage are colocated at the edge — no servers to provision, with cron triggers providing scheduled mining. This matches the PRD's deployment and cost constraints (free-tier operability for demonstration).

3. System analysis and design

3.1 Requirements overview

Derived from the PRD: a unified event backbone (§4); chatbot with policy RAG and transactional leave intents (§5.1); attendance monitoring with anomaly detection (§5.2); a leave state machine with role- and directorate-scoped approvals (§5.3); process discovery, bottleneck analysis, and conformance checking (§6.1–6.3); rule-based decision support (§6.4); measurable success criteria (§13).

3.2 Architecture

A single Worker hosts modular Hono sub-applications (`intelligence` , `attendance` , `leave` , `org` , `chatbot` , `stats`) plus ingestion endpoints — split only if a module outgrows it. Source subsystems POST canonical events; everything downstream (models, bottlenecks, conformance, recommendations, dashboards) is derived from the log. The dashboard SPA is served as static assets from the same Worker. The architecture figure (page 2 of the Demonstration Guide) depicts the flow: subsystems → Integration Engine → Unified Event Log → Process Intelligence.

Event model (canonical): { `case_id`, `activity`, `resource`, `timestamp`, `source_system`, `metadata` } — validated with zod at ingestion; `metadata` carries subsystem-specific payloads as JSON.

3.3 Data model (D1)

TABLE	ROLE
<code>events</code>	The unified log: event id, case id, activity, resource, timestamp, source, JSON metadata, ingestion time
<code>departments</code> , <code>employees</code>	Org directory (9 units; 150 staff with roles: <code>staff</code> , <code>line_manager</code> , <code>admin_officer</code> , <code>schedule_officer</code> , <code>hr_officer</code> , <code>director</code>)
<code>attendance_records</code>	Daily rollup per employee (clock in/out, late, anomalies)
<code>leave_requests</code> , <code>workflow_transitions</code>	Live workflow state and full transition history
<code>process_models</code>	Mined DFG + variants per source, versioned by run timestamp
<code>bottlenecks</code>	Transition-duration statistics with SLA flags per run
<code>conformance_results</code>	Deviations with type, description, score per run

Analysis tables are **append-only per mining run**; readers select the latest run, giving free history and idempotent re-runs.

3.4 Key design decisions and rationale

1. **Log as the only source of truth.** Seeded history and live API traffic are indistinguishable downstream — the pipeline dashboard works identically on both.
2. **Enforcement model = audit model.** The conformance checker consumes the same prescribed chains (`prescribedChain(type)`) that the transition endpoint enforces, eliminating model drift.

3. **Rule-based recommendations first.** Deterministic rules over mining results are explainable and testable; the AI narrative layer is explicitly a later phase.
 4. **Hybrid intent routing.** First-person personal-data asks ("how many days was I late") resolve via deterministic keyword rules against the database — the LLM never touches personal records; only policy language and transactions are classified by the small model (llama-3.2-3b).
 5. **Transaction safety.** Leave dates are extracted only from the user's own message, never model-generated; leave creation shares one code path between REST and chat.
 6. **Configurable thresholds.** Bottleneck SLA thresholds live in a KV namespace (bottleneck_thresholds_ms) with code defaults — tunable without redeploy.
 7. **Server-sent events for liveness.** The operations feed streams new events over SSE (with polling fallback), avoiding 5-second polling latency at negligible cost.
-

4. Implementation highlights

4.1 Process discovery (`src/mining/graph.ts`)

Single pass over the chronologically ordered log: node/edge frequencies per source system; variant signatures (activity sequence per case) counted and truncated to the top 10; dependency measure per §2.1. Length-1/2 loop special-casing is deliberately deferred until real data demands it.

4.2 Bottleneck analysis (`src/mining/bottlenecks.ts`)

Transition durations are computed in SQL using window functions (`LEAD(activity) OVER (PARTITION BY source, case ORDER BY timestamp)`), then aggregated in TypeScript: mean, median, P95 (nearest-rank), max. A pair is **flagged** when its median exceeds the source's threshold (defaults: 2 days for leave, 12 hours for attendance; chat gaps never flagged).

4.3 Conformance checking (`src/mining/conformance.ts`)

Terminal traces (completed/rejected; cancellations are legitimate exits) are replayed against the prescribed chain for their type — chosen by content: a trace containing RTDD-only activities is judged against the study chain. Rejected cases are judged only up to the rejecting step. Deviations: `skipped_step` (prescribed activity absent) and `out_of_order`. Score:

```
score = (|expected| - |missing| - [out of order ? 1 : 0]) / |expected|
```

4.4 Leave state machine (`src/lib/workflow.ts` , `src/routes/leave.ts`)

Five steps across two chains. `supervisor_review` forks on leave type; every rule carries a role plus a scope — the requester's unit (supervisors) or a fixed directorate (F&A/RTDD). The transition endpoint enforces role *and* directorate, emits an event per action, and records history; the seeded 10% supervisor-bypass violations are detectable precisely because enforcement and audit agree.

4.5 Chatbot (`src/routes/chatbot.ts` , `src/lib/rag.ts`)

Intent pipeline: keyword rules (attendance/balance lookups) → small-model classification → handlers. Policy answers: RAG per §2.3 with source citations surfaced as chips in the UI. Leave requests: dates parsed from the user message, created via the shared `createLeaveRequest` , immediately visible in the event log.

4.6 Dashboard engineering (`web/`)

React SPA, hash-routed tabs (Operations, Process Intelligence, Decision Support, My Leave). The process map renders as interactive SVG with a BFS-layered layout that terminates on cycles (real mined graphs contain them); bottlenecks overlay as red pills. Case drill-down opens the raw trace from any finding. My Leave provides submission, tracking, and a role-filtered approver inbox for both chains. Decision Support exports to CSV and print/PDF.

5. Data and validation methodology

5.1 Synthetic data generation

`scripts/seed.mjs` replays six months of OHCS operations with a deterministic PRNG (mulberry32, seed 42): ~33,300 events across the three sources for the 150-staff org. Case ids are namespaced by seed; `--reset` wipes all tables FK-safely before regenerating.

5.2 Seeded ground truth

To make detection *measurable*, the generator plants known phenomena:

1. **A slow step:** F&A verification delays of 1–6 days (median $\approx 3.3d$) — the bottleneck detector should flag `supervisor_review` $\rightarrow$ `fa_verification`.
2. **Violations:** 10% of leave cases skip supervisor review — conformance should recover them all (22 of 256 cases).
3. **Department outliers:** e.g. CSC punctuality (13.8% late), PR leave cycle (7.6 days).

Because the answers are known in advance, accuracy is computed against independent SQL ground truth, not self-assessment.

5.3 Validation harness

`scripts/validate.mjs` measures the six PRD §13 metrics against a running instance (§6 below). `npm test` runs 32 tests inside the real Workers runtime (`@cloudflare/vitest-pool-workers`): ingestion validation and auth, the state machine (full F&A chain, RTDD routing, scoping rejections, terminal-state guards), DFG construction, conformance logic, inbox listings, SSE streaming, and the end-to-end mining pipeline.

6. Evaluation

6.1 PRD success metrics (all passing)

METRIC	TARGET	MEASURED
Event capture latency	< 500 ms	p50 $\approx$ 8 ms (local dev)
Workflow variants discovered	≥ 3	9
Bottleneck detection (planted slow step)	F&A verification flagged	<code>supervisor_review</code> $\rightarrow$ <code>fa_verification</code> , median 3.3d, P95 5.7d
Conformance detection rate	> 80% of violations	22/22 = 100%, 0 false positives
Dashboard load time	< 2 s	< 100 ms
Chatbot policy resolution	> 60%	5/5 = 100%

6.2 Discussion

The miner independently rediscovered the organisation's two prescribed leave chains — standard through F&A, study through RTDD — without configuration, which is the core process-mining claim demonstrated end-to-end. Conformance achieves 100% recall at 0 false positives on the seeded violations; the deterministic ground truth (independent SQL audit) guards against circular self-evaluation. Bottleneck detection localises the planted slow step exactly, with statistics overlaid on the discovered map.

6.3 Limitations and threats to validity

- **Synthetic data:** distributions are engineered; real subsystem noise (duplicate taps, clock drift, back-dated entries) is untested. Mitigation: canonical schema validation at ingestion, and the additive-seed design that lets real traffic mix with history.
- **Scale:** mining reads the full log — fine at OHCS scale ($\sim 10^5$ events); the PRD already earmarks monthly partitioning (\$12) for growth.
- **Discovery algorithm:** DFG + dependency measure is "heuristic-miner-lite"; length-1/2 loops and noise filtering are future work. The layout ranks are BFS (shortest-path), not longest-path layering.
- **Conformance reference:** prescribed chains live in code (KV migration planned); only leave is checked — attendance/chat conformance is undefined by policy.
- **Security:** machine endpoints are API-key protected; user identity is a placeholder pending Cloudflare Access/SSO.
- **AI costs:** Workers AI/Vectorize have no local simulation; usage is billed against the free tier (10k neurons/day).

7. Conclusion and future work

The platform demonstrates that a small serverless footprint — one Worker, a SQLite log, and a vector index — is sufficient to deliver genuine process intelligence: discovery, performance analysis, conformance audit, decision support, and grounded conversational access, validated against measurable criteria with a planted ground truth.

Future work, in priority order: per-user identity via Cloudflare Access; queue-based ingestion fanout (Workers Paid); an AI narrative layer over the rule-based recommendations; prescribed-model migration into KV config; loop-aware discovery refinements; and real subsystem connectors (RFID controllers, the HR system of record).

References

1. W. M. P. van der Aalst, *Process Mining: Data Science in Action*, 2nd ed., Springer, 2016.
2. A. J. M. M. Weijters, W. M. P. van der Aalst, A. K. Alves de Medeiros, *Process Mining with the Heuristics Miner Algorithm*, BETA Working Paper 166, TU/e, 2006.
3. IEEE Std 1849-2016, *IEEE Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams*.
4. P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020.
5. Cloudflare Developer Documentation: Workers, D1, Vectorize, Workers AI, R2, KV.

Companion documents: [README.md](#) (build/run), [IIE_PRD.pdf](#) (requirements). All implementation facts cited against the repository as of July 2026.